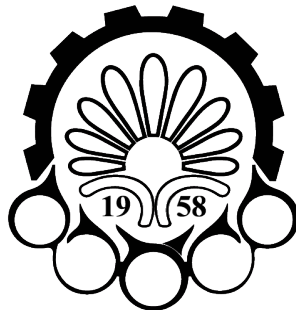


به نام خدا



Amirkabir University of Technology
(Tehran Polytechnic)

دانشکده مهندسی
کامپیوتر و فناوری اطلاعات

مستندات پروژه درس مهندسی نرم افزار ۲

کاوه احمدی ۴۰۱۳۱۹۰۴

آرمینا متقی ۴۰۱۳۱۴۱۸

تیر ۱۴۰۵

۱ Product Vision

پلتفرم End-to-End Ticketing یک سیستم با High Availability برای مدیریت Flash Sales و ترافیک بالا است. هدف سیستم جلوگیری از Downtime و مقیاس پذیری پویا برای توسعه سیستم است.

۱.۱ کاربران هدف

- End-User نیازمند جستجوی سریع رویداد، مشاهده نقشه Real-Time سالن، قرارگیری در Virtual Waiting Room و دریافت QR Code.
- Admin نیازمند طراحی سالن چندبخشی، تنظیم Dynamic Pricing و مشاهده Dashboards تحلیلی در لحظه.

۲.۱ معیارهای کلیدی عملکرد

- آ) Checkout Completion Rate بررسی درصد موفقیت پرداخت پس از قفل صندلی.
- ب) Seat Lock Expiration Rate بررسی نرخ انقضای قفل های موقت Redis.
- ج) Latency حفظ زمان پاسخگویی زیر یک ثانیه و پایداری Uptime بالا.
- د) Gross Merchandise Value پیگیری درآمدزایی سیستم به صورت زنده.

۲ Risk Mitigation Analysis

۱.۲ ریسک اول Flash Traffic Spikes

شرح ریسک: هجوم همزمان ربات ها و کاربران به API Gateway و خطر از کار افتادن Event Catalog.
راهکار مهندسی:

- اجرای Virtual Waiting Room برای Traffic Throttling.
- تنظیم Rate Limiting در لایه Edge.
- استفاده از Auto-scaling در Kubernetes.

۲.۲ ریسک دوم Race Conditions

شرح ریسک: درخواست همزمان برای یک صندلی و خطر Double-Booking.
راهکار مهندسی:

- پیاده سازی Distributed Locks توسط Redis با Time-To-Live مشخص.
- استفاده از Optimistic Concurrency Control در PostgreSQL.

۳.۲ ریسک سوم Payment Gateway Timeout

شرح ریسک: قطعی درگاه بانکی در حین Checkout و گیر کردن صندلی در حالت رزرو. راهکار مهندسی:

- اجرای Saga Pattern و تراکنش‌های Compensation.
- آزادسازی خودکار قفل‌ها و بازگشت صندلی به سیستم در صورت دریافت خطا.

۴.۲ ریسک چهارم Network Partitioning

شرح ریسک: قطعی ارتباط بین سرویس‌ها مانند عدم ارسال SMS تایید به مشتری. راهکار مهندسی:

- استفاده از Asynchronous Messaging توسط Message Brokers.
- ارسال پیام‌های ناموفق به Dead-Letter Queue جهت تلاش مجدد.

۳ Product Vision

پلتفرم End-to-End Ticketing یک سیستم با High Availability برای مدیریت Flash Sales و ترافیک بالا است. هدف سیستم جلوگیری از Downtime و مقیاس‌پذیری پویا برای توسعه سیستم است.

۱.۳ کاربران هدف

- End-User نیازمند جستجوی سریع رویداد مشاهده نقشه Real-Time سالن قرارگیری در Virtual Wait-ing Room و دریافت QR Code.
- Admin نیازمند طراحی سالن چندبخشی تنظیم Dynamic Pricing و مشاهده Dashboards تحلیلی در لحظه.

۲.۲ معیارهای کلیدی عملکرد

- آ) Checkout Completion Rate بررسی درصد موفقیت پرداخت پس از قفل صندلی.
- ب) Seat Lock Expiration Rate بررسی نرخ انقضای قفل‌های موقت Redis.
- ج) Latency حفظ زمان پاسخگویی زیر یک ثانیه و پایداری Uptime بالا.
- د) Gross Merchandise Value پیگیری درآمدزایی سیستم به صورت زنده.

۴ Risk Mitigation Analysis

۱.۴ ریسک اول Flash Traffic Spikes

شرح ریسک هجوم همزمان ربات‌ها و کاربران به API Gateway و خطر از کار افتادن Event Catalog راهکار مهندسی

- اجرای Virtual Waiting Room برای Traffic Throttling.
- تنظیم Rate Limiting در لایه Edge.
- استفاده از Auto-scaling در Kubernetes.

۲.۴ ریسک دوم Race Conditions

شرح ریسک درخواست همزمان برای یک صندلی و خطر Double-Booking راهکار مهندسی

- پیاده‌سازی Distributed Locks توسط Redis با Time-To-Live مشخص.
- استفاده از Optimistic Concurrency Control در PostgreSQL.

۳.۴ ریسک سوم Payment Gateway Timeout

شرح ریسک قطعی درگاه بانکی در حین Checkout و گیر کردن صندلی در حالت رزرو. راهکار مهندسی

- اجرای Saga Pattern و تراکنش‌های Compensation.
- آزادسازی خودکار قفل‌ها و بازگشت صندلی به سیستم در صورت دریافت خطا.

۴.۴ ریسک چهارم Network Partitioning

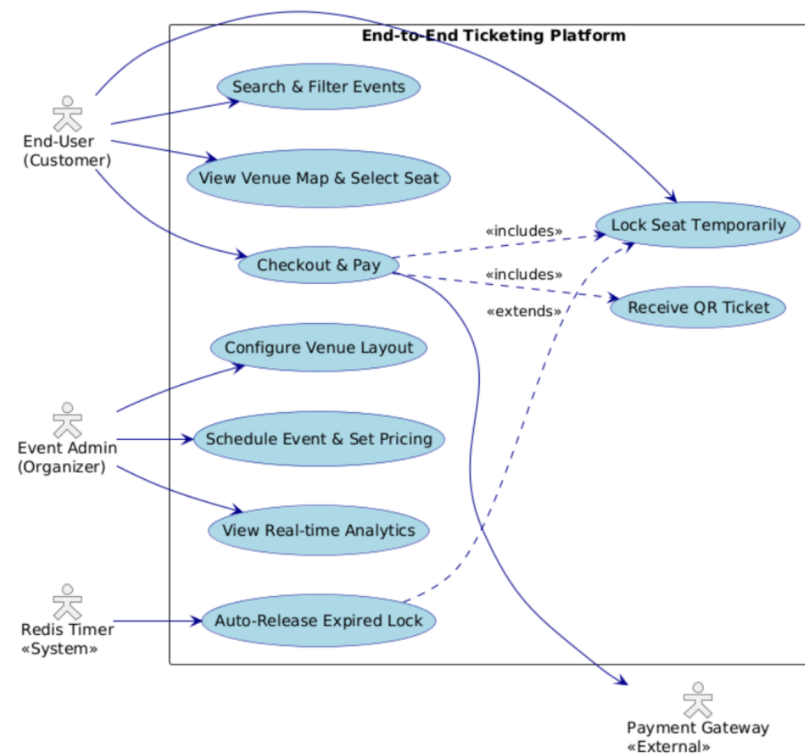
شرح ریسک قطعی ارتباط بین سرویس‌ها مانند عدم ارسال SMS تایید به مشتری. راهکار مهندسی

- استفاده از Asynchronous Messaging توسط Message Brokers.
- ارسال پیام‌های ناموفق به Dead-Letter Queue جهت تلاش مجدد.

۵ Diagrams

۱.۵ Use Case Diagram

این بخش شامل Use Case Diagram سیستم است که تعامل بین Actors و Use Cases را نشان می‌دهد.



۱.۱.۵ تحلیل اکترها

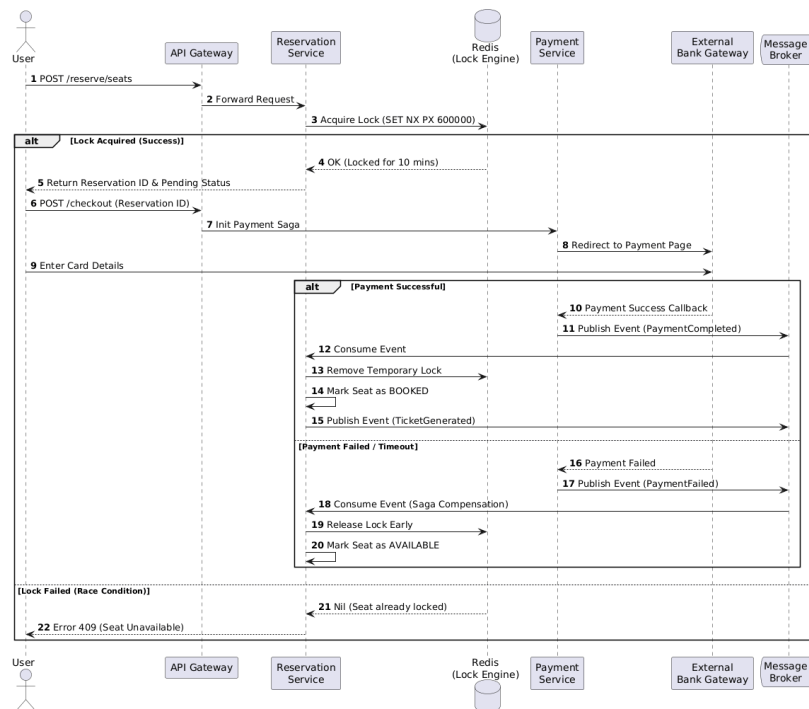
- End-User خریدار بلیت که عملیاتی مانند جستجو انتخاب صندلی و Checkout را انجام می دهد.
- Event Admin مدیر سیستم که وظیفه Configure Venue Layout و بررسی Analytics را بر عهده دارد.
- Payment Gateway یک External Actor برای پردازش تراکنش های مالی.
- Redis Timer یک System Actor برای مدیریت زمان انقضای قفل صندلی ها.

۲.۱.۵ تحلیل روابط

- Includes کاربر برای تکمیل Checkout حتما باید مرحله Lock Seat Temporarily را انجام دهد.
- Extends در صورت عدم پرداخت در زمان مقرر سیستم به صورت خودکار Auto-Release Expired Lock را اجرا می کند تا صندلی آزاد شود.

۲.۵ Sequence Diagram

این بخش Sequence Diagram فرآیند رزرو و پرداخت بلیت را نشان می دهد که تعامل بین کامپوننت ها را در طول زمان مشخص می کند.



۱.۲.۵ تحلیل جریان اصلی

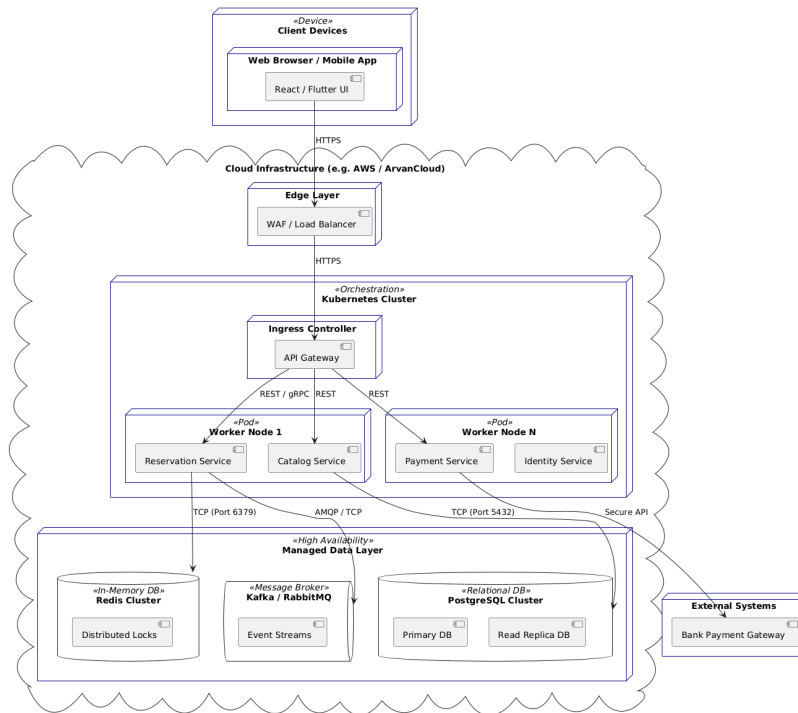
- Lock Acquisition کاربر درخواست POST /reserve/seats را ارسال می کند و Reservation Service با دستور SET NX PX 600000 صندلی را در Redis برای ده دقیقه قفل می کند.
- Payment Saga پس از دریافت Reservation ID کاربر عملیات Checkout را آغاز کرده و به صفحه پرداخت هدایت می شود.

۲.۲.۵ تحلیل سناریوها

- Payment Successful پس از موفق Callback Payment Service رویداد PaymentCompleted را منتشر می کند سپس قفل موقت حذف شده و صندلی به وضعیت BOOKED تغییر می کند و رویداد TicketGenerated منتشر می شود.
- Payment Failed در صورت شکست یا Timeout رویداد PaymentFailed منتشر شده و Saga Compensation اجرا می شود تا قفل زودتر آزاد و صندلی به وضعیت AVAILABLE برگردد.
- Race Condition اگر صندلی از قبل قفل شده باشد Redis مقدار Nil برمی گرداند و سیستم خطای Error 409 را نمایش می دهد.

۳.۵ Deployment Diagram

این بخش Deployment Diagram است که نحوه استقرار کامپوننت ها بر روی زیرساخت فیزیکی و ابری را نمایش می دهد.



۱.۳.۵ تحلیل لایه‌ها

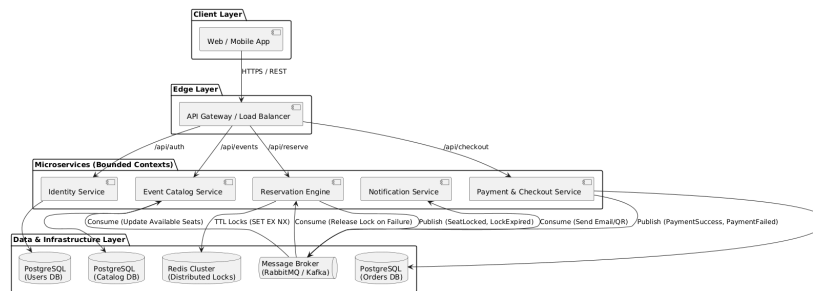
- Client Devices شامل Web Browser و Mobile App است که با استفاده از React / Flutter UI پیاده‌سازی شده و از طریق HTTPS ارتباط برقرار می‌کند.
- Edge Layer شامل WAF / Load Balancer برای مدیریت ترافیک ورودی و امنیت است.
- Kubernetes Cluster یک لایه Orchestration که شامل Ingress Controller و Worker Node های مختلف برای اجرای سرویس‌ها است.
- Managed Data Layer با قابلیت High Availability شامل Redis Cluster و Kafka / RabbitMQ و PostgreSQL Cluster است.

۲.۳.۵ تحلیل ارتباطات

- Redis Connection سرویس‌ها از طریق TCP Port 6379 به Distributed Locks دسترسی دارند.
- Database Connection ارتباط با PostgreSQL از طریق TCP Port 5432 انجام می‌شود.
- External API پرداخت‌ها از طریق Secure API به Bank Payment Gateway متصل می‌شوند.

۴.۵ Component Diagram

این بخش Component Diagram است که کامپوننت‌های اصلی سیستم و ارتباط بین Microservices را در قالب Bounded Contexts نشان می‌دهد.



۱.۴.۵ تحلیل لایه‌ها

- Client Layer شامل Web / Mobile App است که از طریق HTTPS / REST با سیستم ارتباط برقرار می‌کند.
- Edge Layer شامل API Gateway / Load Balancer است که درخواست‌ها را به سرویس‌های مربوطه هدایت می‌کند.
- Microservices شامل Identity Service و Event Catalog Service و Reservation Engine و Notification Service و Payment & Checkout Service است.
- Data & Infrastructure Layer شامل چند PostgreSQL مجزا و Redis Cluster و Message Broker است.

۲.۴.۵ تحلیل تعاملات رویدادمحور

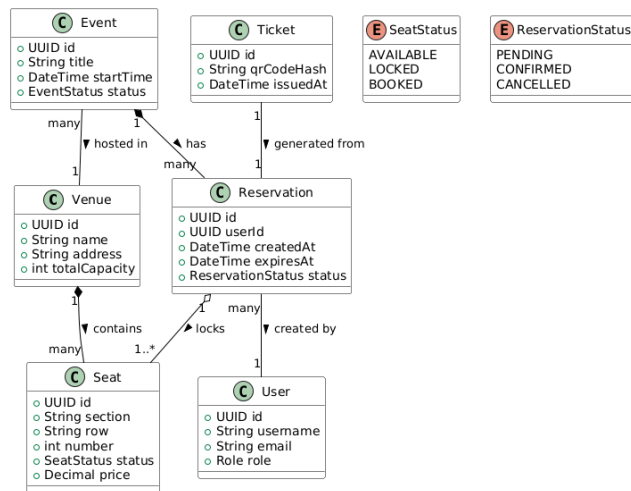
- Distributed Locks موتور رزرو از TTL Locks با دستور SET EX NX در Redis استفاده می‌کند.
- Event Publishing رویدادهای SeatLocked و LockExpired و PaymentSuccess و PaymentFailed از طریق Message Broker منتشر می‌شوند.
- Event Consuming سرویس‌ها رویدادها را برای Update Available Seats و Release Lock on Failure و Send Email/QR مصرف می‌کنند.

۵.۵ Class Diagram

این بخش Class Diagram است که ساختار کلاس‌ها و روابط بین موجودیت‌های اصلی سیستم را نمایش می‌دهد.

۱.۵.۵ تحلیل کلاس‌ها

- Event رویداد اصلی که شامل title و startTime و status است و در یک Venue برگزار می‌شود.
- Venue محل برگزاری که شامل name و address و totalCapacity است و شامل چندین Seat می‌باشد.
- Reservation رزرو که با userId و createdAt و expiresAt مشخص می‌شود و صندلی‌ها را قفل می‌کند.
- Ticket بلیت نهایی که شامل qrCodeHash است و از یک Reservation تولید می‌شود.
- Seat صندلی که شامل section و row و number و price است.
- User کاربر که شامل username و email و role است.

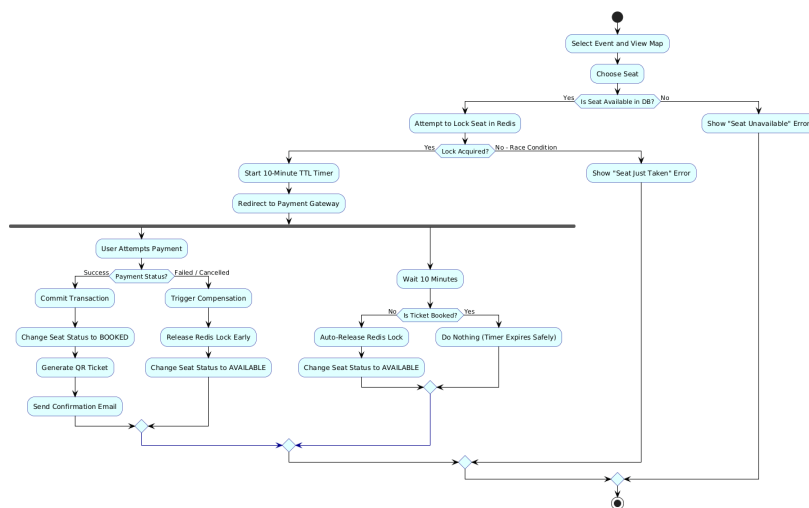


۲.۵.۵ تحلیل Enumerations

- SeatStatus شامل مقادیر AVAILABLE و LOCKED و BOOKED است.
- ReservationStatus شامل مقادیر PENDING و CONFIRMED و CANCELLED است.

۶.۵ Activity Diagram

این بخش Activity Diagram است که جریان کاری فرآیند رزرو صندلی و پرداخت را همراه با شاخه‌های تصمیم‌گیری نشان می‌دهد.



۱.۶.۵ تحلیل جریان انتخاب صندلی

- Seat Availability پس از Choose Seat سیستم بررسی می کند که آیا صندلی در پایگاه داده موجود است در غیر این صورت خطای Seat Unavailable نمایش داده می شود.
- Lock Attempt در صورت موجود بودن سیستم تلاش می کند صندلی را در Redis قفل کند و اگر Race Condition رخ دهد خطای Seat Just Taken نمایش داده می شود.
- TTL Timer پس از قفل موفق یک تایمر ده دقیقه ای شروع شده و کاربر به Payment Gateway هدایت می شود.

۲.۶.۵ تحلیل جریان پرداخت

- Payment Success در صورت موفقیت تراکنش Commit شده و صندلی به BOOKED تغییر می کند سپس QR Ticket تولید و ایمیل تأیید ارسال می شود.
- Payment Failed در صورت شکست یا لغو Compensation فعال شده و قفل زودتر آزاد و صندلی به AVAILABLE برمی گردد.
- Timeout Handling در صورت گذشت ده دقیقه اگر بلیت Booked نشده باشد قفل Redis به صورت خودکار آزاد می شود در غیر این صورت تایمر بدون تأثیر منقضی می شود.